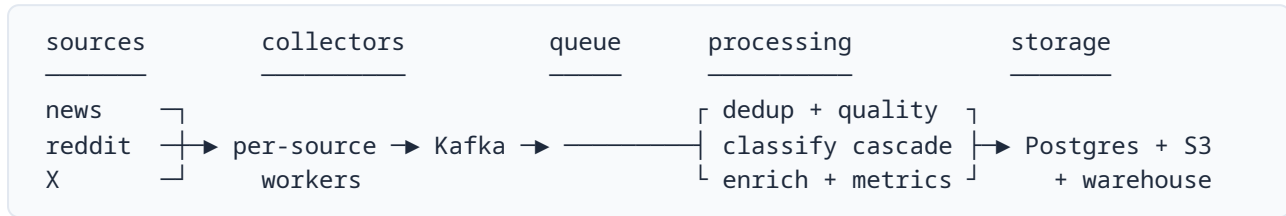


Part 3 — Automating Daily Data Collection

How I would collect mentions from news sites, Reddit, and X every day, store them, keep them clean, and scale the whole thing. The classification work from Part 1 becomes the processing step; this is the plumbing around it.



Collecting

Each source gets a small adapter that pulls its data and maps it to one schema: `{id, date, source, source_tier, url, title, text, channel, reach?}`.

News. Use structured feeds wherever they exist (RSS, Atom, sitemaps), plus Google News RSS and GDELT for discovery and a news API such as NewsAPI as a backstop. Sites without feeds get a Scrapy spider, upgraded to Playwright when the page needs JavaScript. Prefer feeds and APIs over scraping; they break less and stay on the right side of terms of service.

Reddit. The official API through PRAW, searching brand keywords across the relevant subreddits (r/MutualFundsIndia, r/IndiaInvestments, and similar) and polling new posts and comments on a schedule.

X / Twitter. The v2 filtered stream or recent-search endpoint with brand keyword and cashtag rules. Because X access is expensive and rate-limited, keep a paid provider (Apify, Brandwatch) behind the same adapter so the rest of the system doesn't care which one is live.

A scheduler (Airflow or a cloud equivalent) runs the collectors daily, or continuously for X. The brand-match rule from Part 1's relevance filter runs at the edge so obvious noise never enters the queue.

Storing

Three layers, each with a clear job:

- **Raw landing zone (S3).** Every payload is stored as-is, partitioned by source and date. This is the audit trail and it makes the pipeline replayable: re-run classification without re-scraping.
- **Operational store (Postgres).** Normalised mentions and their labels, partitioned by date and upserted by content key. A pgvector column holds the embeddings used for dedup and semantic search.
- **Warehouse (BigQuery or Snowflake).** Aggregates that feed the dashboard and BI tools once volume outgrows Postgres.

Each label is stamped with the model and prompt version, so results are reproducible and a model change can be back-filled cleanly.

Keeping it clean

Deduplication reuses Part 1's logic at stream scale: canonicalise the URL, hash the content for exact matches, check embedding similarity against a rolling window for near-duplicates, and match headlines

across outlets for syndication. Upserting by content key makes re-runs idempotent.

Before anything is classified it passes quality gates: schema validation, language detection, dead-link checks, spam and bot heuristics (account age, repost rate), and the brand-relevance filter. PII is masked before any text leaves the box. Confidence and sentiment distributions are tracked over time, so model or data drift shows up as a measurable shift rather than a surprise.

Scaling

The one idea that makes this affordable is the classification cascade. The cheap local model handles the confident majority; only low-confidence records reach the LLM, which caps cost as volume grows. Batch the escalated calls and cache prompts to push it further.

Everything else is standard horizontal scaling. A queue between collection and processing lets the two sides scale on their own, so a slow scraper never blocks classification. Collectors and classifiers run as autoscaling containers. Each source handles its own rate limits with backoff, and only new or changed records are processed each cycle, with dedup running against a bounded window rather than all history.

Trade-offs

- **X access is the hard constraint.** The official API is costly and limited, which pushes toward paid providers and a coverage-versus-cost decision.
- **Scraping is brittle and legally sensitive.** Sites change and block bots, so it needs upkeep; lean on official feeds and respect robots.txt and terms of service.
- **LLM cost versus accuracy.** The cascade keeps it in check, but a higher escalation rate buys accuracy at a price. The confidence threshold is the dial.
- **Reach data is uneven** across sources, so impact-weighted metrics stay approximate.
- **Real-time versus batch.** Streaming X raises cost and complexity; daily batch is cheaper and enough for reputation reporting. Start with batch and add streaming only where latency actually matters.